PCCPU

James Augustus

Bir cümleyle

PCCPU, hesap yapmayan ve işi dağıtan ayrı bir denetleyici birimi. Üzerinde çalıştığım bir tasarım. Henüz bitmedi ve Hunter'ın tek çekirdekli hâline yerleştirmedim.

Fikir

Eski CT-CPU tasarımımda 4096 çekirdek vardı. Bunlara 4097. bir çekirdek ekledim ve ona PCCPU (Program Counter CPU) dedim. Hesap yapmaz, düdük çalar. Ritmi belirler, yani saati. Sırayı tutar, yani sayacı. Kime ne gideceğini söyler, yani adresi. Komutun içeriğini bilmek zorunda değil, yalnız dağıtır. Bir trafik polisi ya da bir iskambil dağıtıcısı gibi.

Tasarımda bir yanlışımı düzelttim. İlk başta PCCPU'nun her saat vuruşunda komut beslediğini sanıyordum. Dört bin çekirdeği tek tek beslemek darboğaz olurdu. Doğrusu şu: PCCPU işi başlatır ve geri çekilir, çekirdekler bağımsız çalışır, her birinin kendi sayacı ve kendi döngüsü vardır. PCCPU'ya yalnız dışarıyla bir iş çıktığında dönülür: bir taşma olduğunda, çekirdekler arasında veri geçeceğinde, bir iş bittiğinde ve senkronizasyon gerektiğinde. Düdükçü "başla" der, sporcular kendi çalışmasını yapar. Düdük yalnız çarpışma, zaman ya da değişim için çalar, her hareketi söylemez.

Üç çalışma biçimi

Merkez biçiminde PCCPU her komutu tek tek yollar ve çekirdekler söyleneni yapar. Bu sıralıdır, paralellik yoktur. Bağımsız biçimde PCCPU işi dağıtır ve izler, her çekirdek kendi programını çalıştırır. Küp ve lider biçiminde çekirdekler küplere ayrılır, PCCPU yalnız liderlerle konuşur ve işçi çekirdekler yalnız hesap yapar.

Bence tek biçim seçmek yanlış, üçü birlikte çalışmalı. Sistem açılınca veriyi merkez biçimiyle yüklerim, sonra küpleri kurarım, boştaki çekirdekler bağımsız çalışır ve PCCPU hangisinin ne zaman etkin olacağına karar verir.

Henüz açık olanlar

Programın nerede tutulacağı açık: registerlarda mı, her çekirdekte küçük bir komut tamponunda mı, yoksa PCCPU'dan yayınla mı. PCCPU'nun kendisinin nereye oturacağı da açık: ızgaranın dışına mı, içine mi, yoksa yazılımla mı. İçinde ne olacağını tartışıyorum: program sayacı, adres çözücü, komut çözücü ve belki çekirdeklerin meşgul mü boş mu olduğunu tutan bir tablo. Birden fazla PCCPU gerekir mi, buna şimdilik tek PCCPU ile başlayıp darboğaz çıkarsa bakacağım.

Hunter bugün tek çekirdekli. Orada komut kaynağının ve program sayacının yeri henüz belli değil. Eski PCCPU'yu bu boşluğa bitmiş bir parça gibi yerleştirmiyorum. Bu belge tasarımın şimdiki durumunu anlatıyor, test edilmiş bir devre göstermiyor.

Hücre üzerinden işlem modeli

Sayıları, görevleri ve denetleyici durumunu ortak bir adres düzeninde göstermek için küçük bir C modeli hazırladım. 1000 ve 1001 adreslerindeki iki işçi, C(9,3) = 84 ve C(9,4) = 126 değerlerini üretir. 1002 adresindeki işçi ikisini bekler, sonra toplamı 1003 adresine yazar. Sonuç C(10,4) = 210'dur. 1004 adresinde denetleyici durumu vardır.

Bu adresler kayıt etiketidir, gerçek RAM bayt adresi değildir. Aritmetik işçi yordamlarında yapılır. Denetleyici hangi işin hazır ve hangisinin tamamlanmış olduğunu izler. Örnekteki tamamlanan iş sayacı bir program sayacı veya zaman ölçümü değildir. İşçi sayısının üç olması da üç fiziksel çekirdek çalıştığı anlamına gelmez. C programı sıralı bir yazılım modelidir.

Bu modeli ve bütün testlerini sade açıklamada adım adım anlattım. C kaynak dosyası ve çalıştırma betiği depodadır.

Matematiksel dayanak: Pascal satırı ve N(p)

Pascal katsayıları için C(n,k) = C(n,n-k) simetrisi vardır. Ayrıca her n ≥ 0 için

```
(x+1)^n = Σ C(n,k) x^k, k = 0,...,n
```

eşitliği geçerlidir. Bu, simetrik katsayıları iki farklı üs konumuna bağlamanın dayanağıdır. Eşit katsayı, eşit ağırlıklı terim demek değildir. C(n,k) x^k ve C(n,k) x^(n-k) genel olarak farklı değerlerdir.

Bir sayının hanelerini doğrudan Pascal katsayıları olarak okumak için x = b ≥ 2 bir tamsayı tabanı seçilir. En büyük katsayı c(n) = C(n,⌊n/2⌋) olsun. Eldesiz hane okuması ancak c(n) < b olduğunda geçerlidir.

Mersenne tabanında b = 2^p - 1, p ≥ 2 için

```
N(p) = max { n ≥ 0 : C(n,⌊n/2⌋) < 2^p - 1 }
```

olarak tanımlıyorum. Eldesiz satırlar 0 ile N(p) arasındadır. Dolayısıyla simetrik katsayı kaydını doğrudan simetrik hane kaydı olarak kullanmak için n ≤ N(p) gerekir. N(p), bu kullanımın matematiksel sınırıdır. Thread zamanlamasını, RAM yerleşimini veya donanım hızını belirlemez.

p = 3 için b = 7 ve N(3) = 4 olur. Çünkü dördüncü satırın merkezi 6 < 7, beşinci satırın merkezi 10 ≥ 7'dir. Dördüncü satır [1,4,6,4,1], 4096 sayısının 7 tabanındaki haneleridir. Beşinci satırın katsayıları hâlâ simetriktir ama elde taşındıktan sonraki haneler [1,6,4,3,5,1] olur. Hane simetrisi artık kullanılamaz.

84 + 126 = 210 işlemi tamsayı hesabı olarak geçerlidir. Fakat b = 127 tabanında N(7) = 9 olduğundan, sonuç satırı n = 10 artık eldesiz değildir. p = 8, b = 255 seçildiğinde merkez 252 < 255 olur ve N(8) = 10 sınırına kadar haneler korunur. Bu yüzden hesap sonucunun doğru olması ile seçilmiş tabanda eldesiz gösterilebilmesi ayrı kontrollerdir.

Bu bölüm, Pascal araştırmamın taban gösterimine dayanır. Yayımdaki Türkçe matematik notu belirli satır seçimini ve eldesiz koşulu açıklar. Buradaki N(p) tanımı, sabit tabanda bu koşulu sağlayan son satırı belirtir. Simetri, Pascal katsayılarının özelliğidir. N(p) ise bu katsayıların hane olarak okunabileceği aralığı sınırlar.

Bir thread, simetrik iki konum

Bir thread kaydının aynı satırdaki iki simetrik konumu birlikte sahiplenmesini öneriyorum. Burada thread, mantıksal görev kimliğidir. İşletim sistemi thread'i veya fiziksel çekirdek olduğu varsayılmaz. Derece n = 4 için örnek şöyledir:

Kayıt	Ortak katsayı	Birinci konum	Simetrik konum
thread_no_1	1	x^0	x^4
thread_no_2	4	x^1	x^3
thread_no_3	6	x^2	Aynı merkez konumu

Böylece thread_no_1, x^0 ve x^4 değerlerini ayrı alanlarda tutabilir. Üs 0 için değer 1'dir. Üs 4 için değer x^4'tür. Ortak katsayı 1 olduğu için katsayıyı iki kez saklamak gerekmez. Ancak iki üs değerinin aynı olduğu söylenemez.

Örneğin x = 7 için thread_no_1'de kuvvet değerleri 1 ve 2401'dir. thread_no_2'de kuvvetler 7 ve 343, katsayıyla çarpılmış katkılar 28 ve 1372'dir. Merkez kaydının katkısı 6 × 49 = 294'tür. Toplam 1 + 2401 + 28 + 1372 + 294 = 4096 olur. Merkez bir kez eklenir.

Gösterim olarak `T(n,k)[a]` kullanılabilir. k, sağdan sayılan üs konumudur ve $0 \le k \le \lfloor n/2 \rfloor$ koşulunu sağlar. a yalnız 0 veya 1 değerini alır. a = 0 için üs k, a = 1 için üs n-k seçilir. Böylece `T(4,0)[0]` sağ uçtaki x^0 konumunu, `T(4,0)[1]` sol uçtaki x^4 konumunu seçer. Bu gösterim çalışma önerisidir. Önceki N(p) işlevinin veya henüz kesinleşmemiş çok parametreli N gösteriminin anlamını değiştirmez.

C(4,1) = C(4,3) = 4 eşitliği, iki yerde de 4 katsayısı bulunduğunu söyler. Buradaki işlem 4^1 ve 4^3 değildir. x^1 ve x^3 ağırlıklarıyla çarpılan ortak 4 katsayısı vardır. Katsayının kendisini konum kuvvetine yükseltmek istenirse bunun için ayrı bir işlem türü gerekir.

Simetriyi taşıyan RAM kaydı

Önerilen kayıt, n satırını, k konumunu, kullanılan x değerini veya ona bir başvuruyu ve ortak katsayıyı tutar. İki tarafın açılmış kuvvet değerleri de saklanacaksa iki ayrı değer alanı gerekir. Değerlerin hazır olup olmadığını gösteren durum alanları ayrıca tutulur. Thread kimliği işi sahiplenen kaydı, adres ise o kaydın yerini belirtir.

Alan	Anlam
n, k	Satır ve simetrik çiftin ilk üs konumu
x veya x_ref	Kuvvetin tabanı ya da ona başvuru
coefficient	Ortak C(n,k) katsayısı
power[0], power[1]	Hazırsa x^k ve x^(n-k) değerleri
ready[0], ready[1]	Her tarafın ayrı hazır olma bilgisi
owner	Kaydı sahiplenen mantıksal thread

Kuvvetlerin tamamı yerine n, k, x ve katsayı saklanabilir. O zaman değerler istenince hesaplanır. İki tam kuvvet değeri saklanırsa onları üretme ve bellekte tutma maliyeti vardır. Bu iki tercih aynı saklama biçimi değildir ve bu belge birini donanım kararı olarak sabitlemez.

n = 4, k = 2 için iki taraf da merkezdeki x^2’ye gider. Bu tek konumdur. Bir kez saklanabilir ve toplamaya bir kez katılır. n = 3 için ise üs çiftleri (0,3) ve (1,2)’dir. Ortadaki katsayılar eşit olsa da iki ayrı konum bulunur. Bu iki katkı da hesaba katılır.

x, n veya k değişirse eski kuvvet değerleri geçerliliğini kaybeder. Tasarımda bir tarafın katsayısı bağımsız değiştirilmek istenirse ortak katsayı bağlantısı da ayrılmalıdır. Aksi hâlde simetri varsayımı yanlış bir eşitlik dayatır. Yazma, geçerlilik ve tamamlanma kuralları RAM protokolünün parçası olacaktır.

Simetrik Pascal katsayılarını saklamak için ⌊n/2⌋ + 1 farklı katsayı yeterlidir. Bu, toplam donanım belleğinin aynı oranda azalacağı anlamına gelmez. İki tarafın açılmış değerleri, adresler ve durum bitleri ek yer tutabilir. Sıradan RAM üzerinde böyle bir kayıt düzeni kurulabilir. Özel RAM donanımı ise ayrıca tasarlanıp sınanmalıdır.

Uygulama durumu ve çalışma

C örneği iki üreticinin ve bir toplayıcının bağımlılığını uygular. 20.100 komşu Pascal toplamını yalnız toplama kullanan bir karşılaştırmayla sınar. Diğer sınır ve durum kontrolleriyle birlikte 60.353 kontrol vardır. Depo kökünden:

```
bash kod/PCCPU/calistir.sh
```

C11 derleyicisi ve GMP geliştirme dosyaları gerekir. Simetrik thread kaydı, çift değerli RAM ve taraf seçimi bu C programına henüz uygulanmadı. Bu ek bölüm onların matematiksel anlamını ve kayıt sözleşmesini tanımlar. Programın mevcut test sayısı bu yeni RAM tasarımının test sonucu olarak okunmamalıdır.

Metin CC BY 4.0. C örneği MIT lisansıyla sunulur.